

Lecture ③①

Recursion

↳ When a function calls itself directly or indirectly

```
solve(int n) {
```

```
            solve(n);    
```

```
}
```

When Recursion is used?

→ Jab Big/Complex problem Ka solⁿ depend Karta Hai same type ki choti problem par Tab Recursion use Karte hai.

Ex:- For finding $2^n \Rightarrow 2 \times 2 \times 2 \times \dots \times 2$
n times
 $\hookrightarrow 2 \times 2^{n-1}$

in form of function $\Rightarrow F(n) = 2 \times F(n-1)$ } (Recursive Relation)

Ex:- For finding factorial $\Rightarrow 5! = 5 \times 4 \times 3 \times 2 \times 1$
 $= 5 \times 4!$

$$\Rightarrow 5! = 5 \times 4!$$

$$\text{So, } n! = n \times (n-1)!$$

$$\underline{F(n) = n \times F(n-1)}$$

In condition me processing hoti hai wo base condition Hogi

Recursion

→ Base Case (Return mandatory)
→ Recursive Relation

Code for factorial

```
int factorial(int n) {
```

```
    if (n == 0) {
```

```
        return 1;
```

```
    }
```

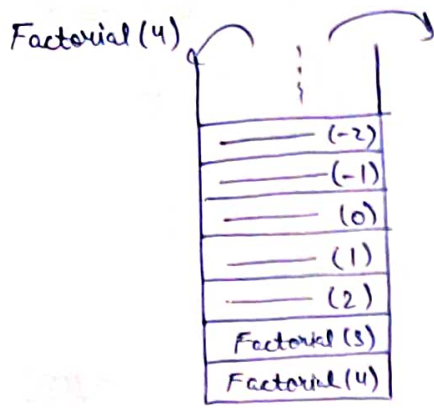
```
    int choti = factorial(n-1);
```

```
    int badi = n * choti;
```

```
    return badi;
```

```
}
```

If no Base case $\Rightarrow$ (Segmentation fault)
Tab memory Bhan jayegi

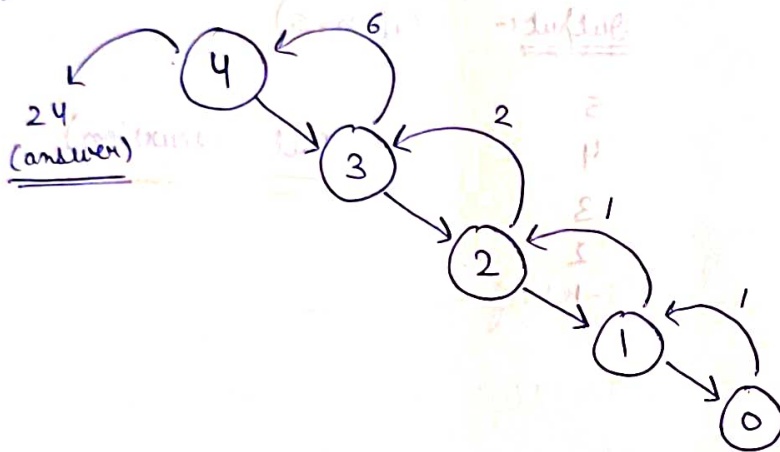


Stack Overflow

Tab segmentation fault
 aa jayega

Recursion Tree

For factorial ($n=4$)



Components of Recursive Functions

func() {

Base Case

Processing

Recursive Relⁿ

}



Tail Recursion

func() {

Base Case

Recursive Relⁿ

Processing

}



Head Recursion

Code for Power of 2

```
int power(int n) {  
    if (n == 0) {  
        return 1;  
    }  
    return 2 * power(n-1);  
}
```

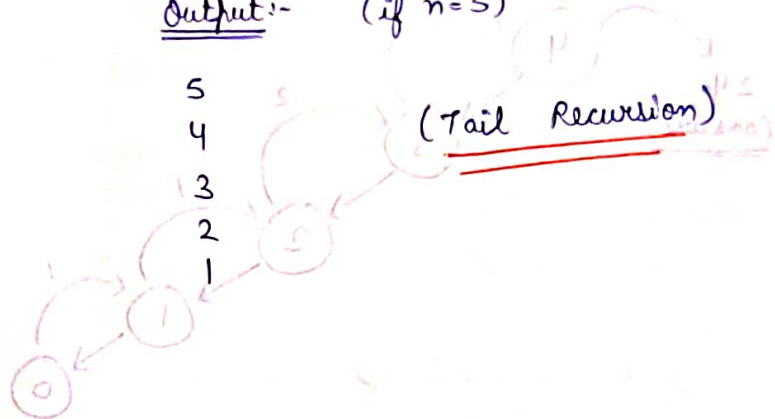
(think recursion) $\Rightarrow$ not easy for
if you could program it



Code for printing Counting

```
void print(int n) {  
    if (n == 0) {  
        return;  
    }  
    cout << n << endl;  
    print(n-1);  
}
```

Output:- (if $n=5$)



```
void print(int n) {
```

```
    if (n == 0) {  
        return;
```

```
    }
```

```
    print(n-1);
```

```
    cout << n << endl;
```

```
}
```

Output:- (if $n=5$)

1
2
3
4
5

(Head Recursion)

Lecture 32

Lecture 509

Qu - Write a recursive function to print n^{th} fibonacci number.

Recursive Relation $\Rightarrow \underline{F(n) = F(n-1) + F(n-2)}$

($n=0$) $\Rightarrow 0$

Base Case

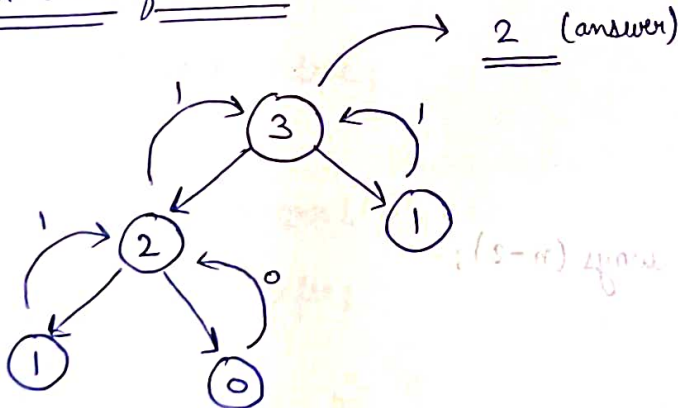
if ($n == 0$)
return 0; } $F(0)$

if ($n == 1$)
return 1; } $F(1)$

Code:-

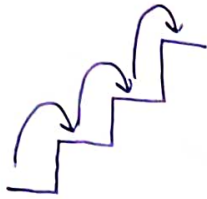
```
int fib(int n) {
    if (n == 0) {
        return 0;
    }
    if (n == 1) {
        return 1;
    }
    int ans = fib(n-1) + fib(n-2);
    return ans;
}
```

Recursion tree for $n=3$



Q- You have been given a number of stairs. Initially, you are at the 0th stair and you need to reach the n th stair. Each time you can either climb one step or two steps. You are supposed to return the number of distinct ways in which you can climb from the 0th step to n th step.

Ex- (n=3)



Base Case
if (n == 0) {
 return 1;
}
if (n == 1) {
 return 1;
}

Recursive Relation $\Rightarrow F(n) = F(n-1) + F(n-2)$

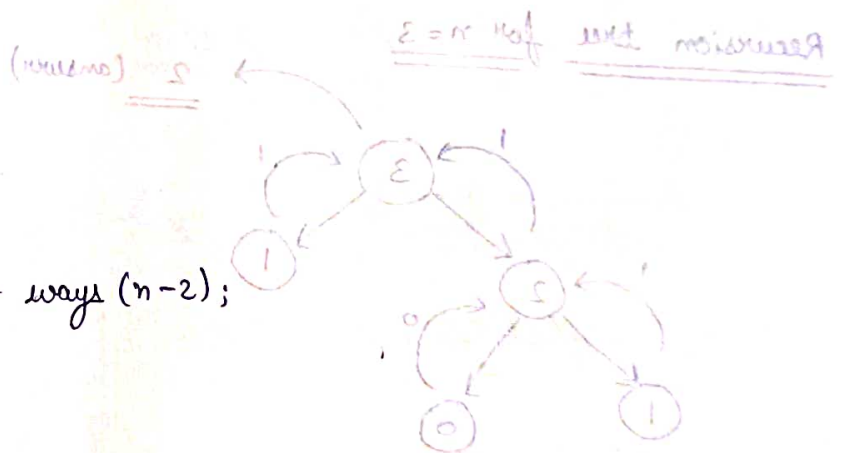
Base Case

```
if (n < 0)
    return 0;

if (n == 0)
    return 1;
```

Code:-

```
int ways (int n) {
    if (n < 0) {
        return 0;
    }
    if (n == 0) {
        return 1;
    }
    int ans = ways(n-1) + ways(n-2);
    return ans;
}
```



Let to see how, Fibonacci. links for number a string need not be - us
 On - You have been given a number and need to find how many ways you can
 say it. For example, 121/P $\rightarrow$ 4124. Here it has 4 ways to reach 121.
 121/P $\rightarrow$ Four One two. Here it has 4 ways to reach 121.
 121/P $\rightarrow$ Four One two. Here it has 4 ways to reach 121.
 121/P $\rightarrow$ Four One two. Here it has 4 ways to reach 121.

```

void sayDigits(int n) {
    if (n == 0) {
        return;
    }
    int digit = n % 10;
    n = n / 10;
    sayDigits(n);
    String arr[10] = {"zero", "one", "two", "three", "four", "five", "six", "seven", "eight", "nine"};
    cout << arr[digit] << " ";
}

```

Lecture 33

Qu- Using Recursion, find if the array is sorted or not.

Base Case

```

if (size == 0 || size == 1)
    return true;

```

```

bool isSorted(int arr[], int n) {

```

```

    if (n == 0 || n == 1) {

```

```

        return true;
    }

```

```

    if (arr[0] > arr[1]) {

```

```

        return false;
    }

```

```

    else {

```

```

        bool ans = isSorted(arr+1, n-1);

```

```

        return ans;
    }
}

```

Ek case khud solve karo

Baki Recursion par chodo do

H.W.
Find the sum of elements of array using Recursion

```
int getSum (int *arr, int size) {
```

```
    if (size == 0) {
```

```
        return 0;
```

```
    }
```

```
    if (size == 1) {
```

```
        return arr[0];
```

```
    }
```

```
    int remainingPart = getSum(arr + 1, size - 1);
```

```
    int sum = arr[0] + remainingPart;
```

```
    return sum;
```

```
}
```

Qu- Implement Linear Search using Recursion

```
bool linearSearch (int arr[], int size, int k) {
```

```
    if (size == 0) {
```

```
        return false;
```

```
    }
```

```
    if (arr[0] == k) {
```

```
        return true;
```

```
    }
```

```
    bool remainingPart = linearSearch(arr + 1, size - 1, k);
```

```
    return remainingPart;
```

```
}
```

```
}
```

Qu- Implement Binary Search using Recursion

```

bool binarySearch(int *arr, int s, int e, int k) {
    if (s > e) {
        return false;
    }
    int mid = s + (e - s) / 2;
    if (arr[mid] < k) {
        return binarySearch(arr, mid + 1, e, k);
    }
    else {
        return binarySearch(arr, s, mid - 1, k);
    }
    if (arr[mid] == k) {
        return true;
    }
}

```

H.W.

Do all questions of Binary Search previously done using Recursion

Lecture 34

Ques- Reverse a string using Recursion

```

void reverse(string str, int i, int j) {
    if (i > j) {
        return;
    }
    swap(str[i], str[j]);
    i++;
    j--;
    reverse(str, i, j);
}

```

Ques- Check Palindrome using Recursion

```

bool checkPalindrome(string str, int i, int j) {
    if (i > j) {
        return true;
    }
    if (str[i] != str[j]) {
        return false;
    }
    else {
        return checkPalindrome(str, i+1, j-1);
    }
}

```

Ques- Calculating Power using Recursion

```

int power(int a, int b) {

```

```

    if (b == 0) {
        return 1;
    }

```

```

    if (b == 1) {
        return a;
    }

```

```

    int ans = power(a, b/2);

```

```

    if (b % 2 == 0) {
        return ans * ans;
    }

```

```

    else {

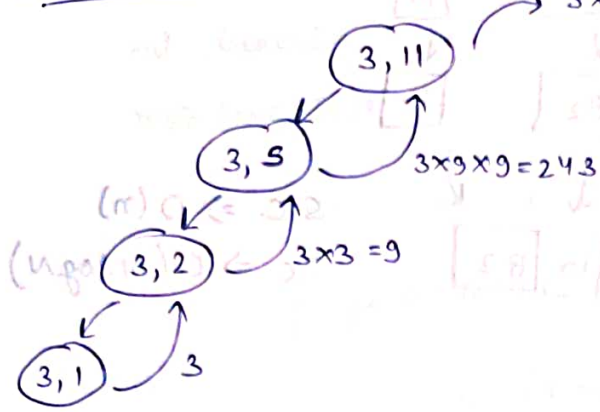
```

```

        return a * ans * ans;
    }
}

```

Recursion tree for 3



Qn- Bubble Sort using Recursion

```
void sortArray(int *arr, int n) {
```

```
    if (n == 0 || n == 1) {
```

```
        return;
```

```
    }
```

```
    for (int i = 0; i < n-1; i++) {
```

```
        if (arr[i] > arr[i+1]) {
```

```
            swap(arr[i], arr[i+1]);
```

```
        }
```

```
    }
```

```
    sortArray(arr, n-1);
```

```
}
```

H.W.

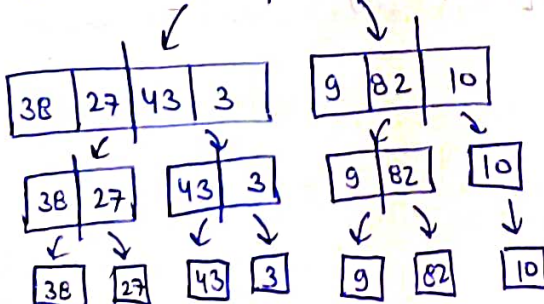
Do Insertion Sort and Selection Sort using Recursion

Lecture 35

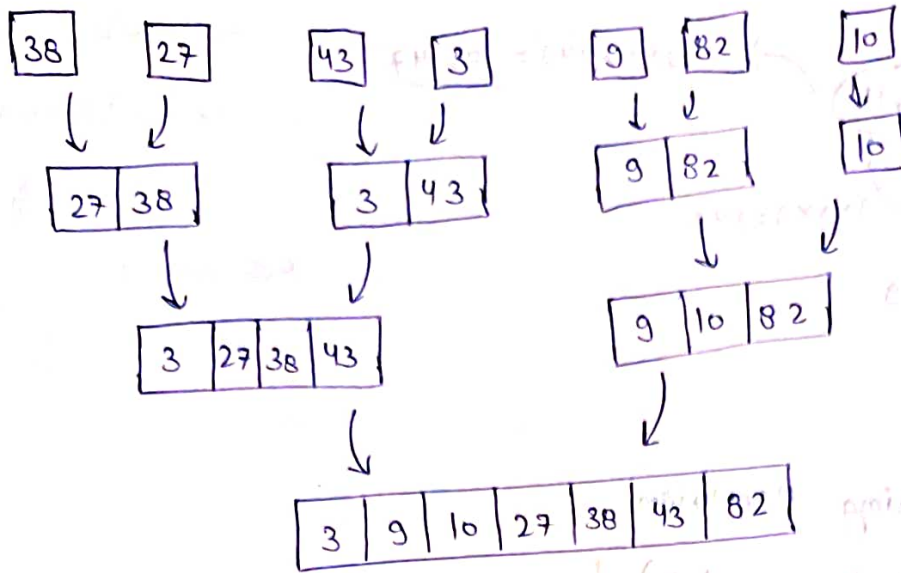
Merge Sort (Recursively Implement)

i/p $\Rightarrow$

38	27	43	3	9	82	10
0	1	2	3	4	5	6



$$\text{mid} = \left(\frac{0+6}{2} \right) = 3 \text{ ref}$$



S.C. $\Rightarrow O(n)$
T.C. $\Rightarrow O(N \log N)$

```
void merge(int *arr, int s, int e) {
```

```
    int mid = s + (e-s)/2;
```

```
    int len1 = mid - s + 1;
```

```
    int len2 = e - mid;
```

```
    int *first = new int[len1];
```

```
    int *second = new int[len2];
```

```
// Copy Values
```

```
    int mainArrayIndex = s;
```

```
    for (int i=0; i<len1; i++) {
```

```
        first[i] = arr[mainArrayIndex++];
```

```
    }
```

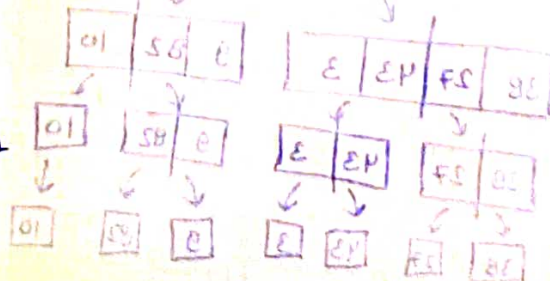
```
    mainArrayIndex = mid + 1;
```

```
    for (int i=0; i<len2; i++) {
```

```
        second[i] = arr[mainArrayIndex++];
```

```
    }
```

```
// merge 2 sorted arrays
```



int index1 = 0;

int index2 = 0;

mainArrayIndex = s;

while (index1 < len1 && index2 < len2) {

if (first[index1] < second[index2]) {

arr[mainArrayIndex++] = first[index1++];

}

else {

arr[mainArrayIndex++] = second[index2++];

}

}

while (index1 < len1) {

arr[mainArrayIndex++] = first[index1++];

}

while (index2 < len2) {

arr[mainArrayIndex++] = second[index2++];

}

}

void mergeSort(int *arr, int s, int e) {

if (s >= e) {

return;

}

int mid = s + (e - s) / 2;

mergeSort(arr, s, mid);

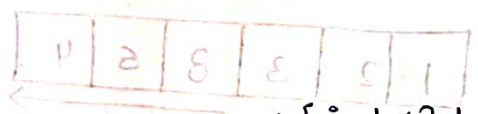
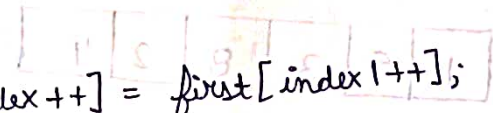
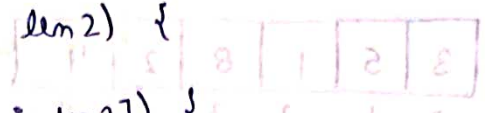
mergeSort(arr, mid + 1, e);

merge(arr, s, e);

}

28/08/2020

28/08/2020



Lecture 36

Quick Sort

(Divide and Conquer Algorithm)

3	5	1	8	2	4
0	1	2	3	4	5

1st step → Ek element uthao aur uska uski right place par rakho such that uske left me jo elements ho vo usse chote ho aur right me jo ho vo usse bade ho this is called partition

Take a pivot

3	5	1	8	2	4
---	---	---	---	---	---

1	5	3	8	2	4
---	---	---	---	---	---

1	2	3	8	5	4
---	---	---	---	---	---

< 3

Partition

→ Take a pivot

→ Count all elements < pivot

→ pivot → s + count

→ Condⁿ:

< a	a	> a
-----	---	-----

```
int partition(int arr[], int s, int e) {
```

```
    int pivot = arr[s];
```

```
    int cnt = 0;
```

```
    for (int i = s + 1; i <= e; i++) {
```

```
        if (arr[i] <= pivot) {
```

```
            cnt++;
```

int pivotIndex = s + cnt;

swap(arr[pivotIndex], arr[s]);

int i = s, j = e;

while (i < pivotIndex && j > pivotIndex) {

while (arr[i] <= pivot) {

i++;

}

while (arr[j] > pivot) {

j--;

}

if (i < pivotIndex && j > pivotIndex) {

swap(arr[i++], arr[j--]);

}

}

return pivotIndex;

}

void quickSort(int arr[], int s, int e) {

if (s >= e) {

return;

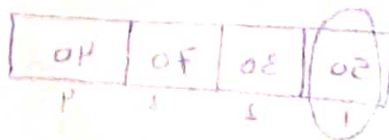
}

int p = partition(arr, s, e);

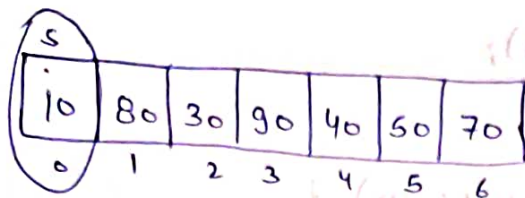
quickSort(arr, s, p-1);

quickSort(arr, p+1, e);

}



Dry Run



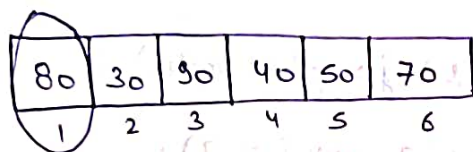
pivot = 10

cnt = 0

pivotIndex = s + cnt
= 0

return 0; (return pivotIndex)

Right wala part sort karo

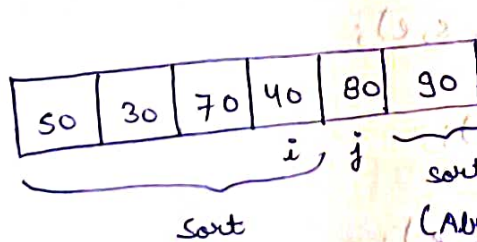
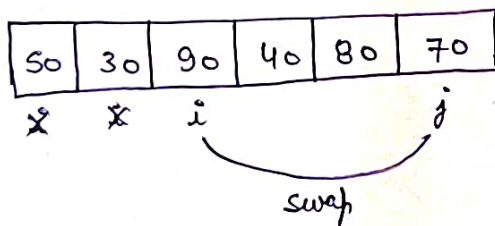


pivot = 80

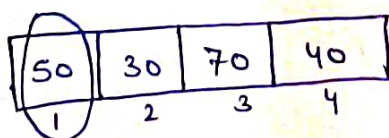
cnt = 4

pivotIndex = s + cnt
= 1 + 4 = 5

swap (arr[1], arr[5]);



Left wala part sort karo

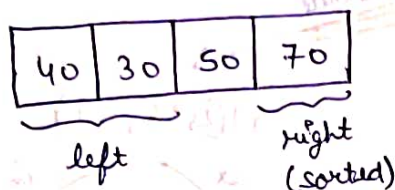
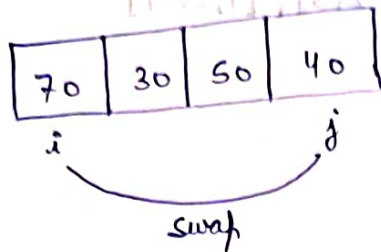


pivot = 50

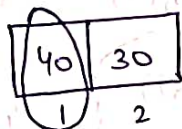
cnt = 2

pivotIndex = s + cnt
= 1 + 2 = 3

Now, after swap



left wala part sort karo

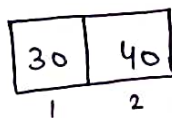


pivot = 40

cnt = 1

pivotIndex = s + cnt
= 1 + 1 = 2

Now, after swap



Now, return pivotIndex i.e. 2

H.W.

Read GFG article in description, Check if quick sort is in-place and stable

S.C. $\Rightarrow O(n)$

T.C. $\Rightarrow O(n \log n) \rightarrow$ (avg. & best case)

Worst case $\Rightarrow O(n^2)$

Lecture 37

LeetCode 78

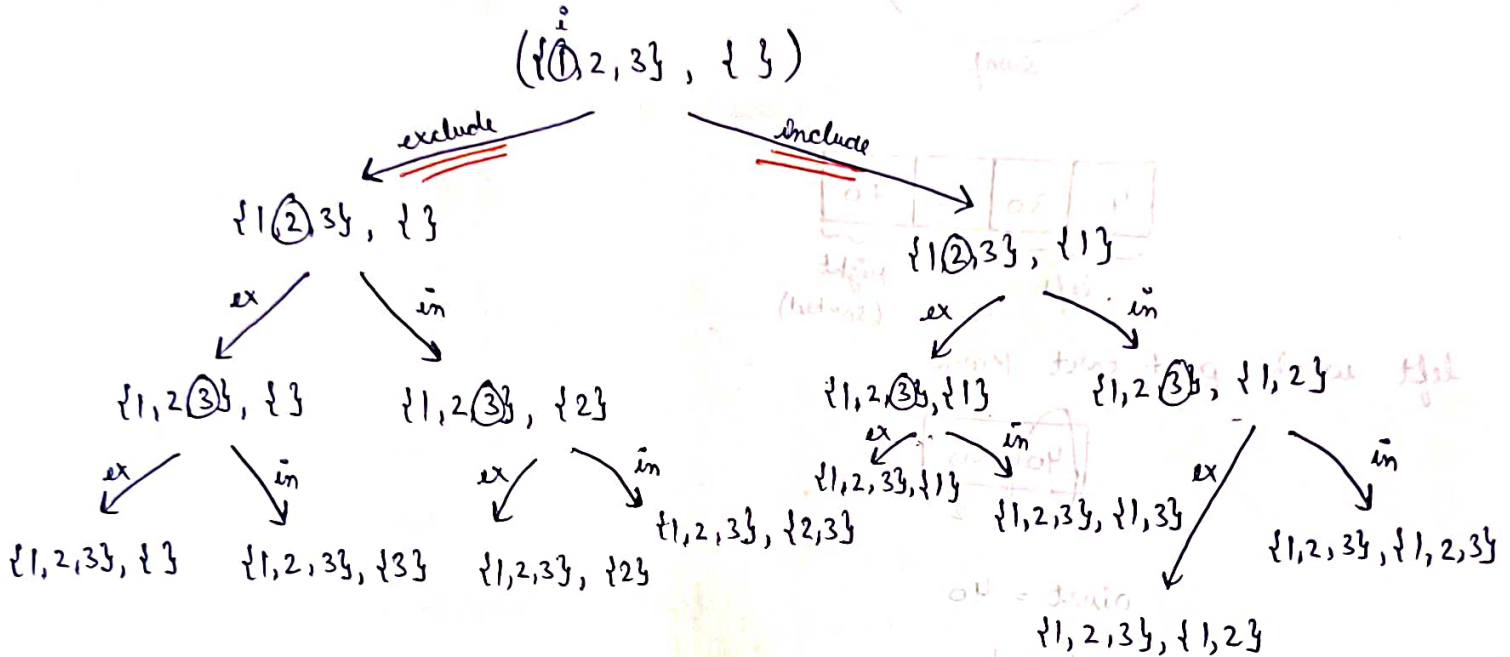
Ques Given an integer array of unique elements, return all possible subsets (the power set).
 → set of all subsets

i/p $\Rightarrow [1, 2, 3]$

o/p $\Rightarrow [[], [1], [2], [1, 2], [3], [1, 3], [2, 3], [1, 2, 3]]$

Soln

no. of elements in power set $= 2^n$



void solve(vector<int> nums, vector<int> output, int index, vector<vector<int>>& ans){

// base case

if (index >= nums.size()) {

ans.push_back(output);

return;

}

// exclude

solve(nums, output, index + 1, ans);

// include

int element = nums[index];

output.push_back(element);

solve(nums, output, index + 1, ans);

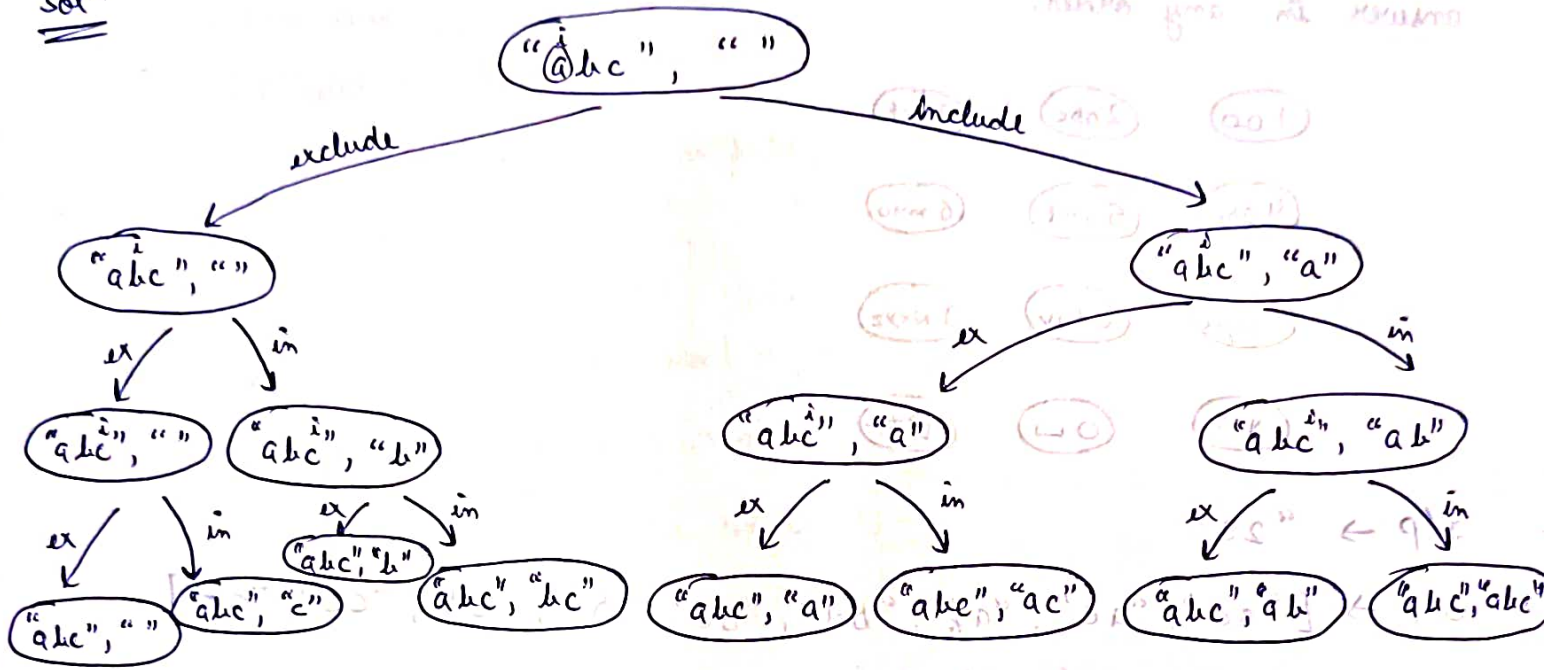
}

Q- You are given a string str containing lowercase english letters from a to z inclusive. Your task is to find all non-empty possible subsequences of str.

i/p $\rightarrow$ abc

o/p $\rightarrow$ a ab abc ac b bc c

Solⁿ



```
void solve (string str, string output, int index, vector<string> &ans) {
```

// base case

```
if (index >= str.length()) {
    if (output.length() > 0) {
        ans.push_back(output);
```

return;

// exclude

```
solve (str, output, index+1, ans);
```

// include

```
char element = str[index];
```

```
output.push_back(element);
```

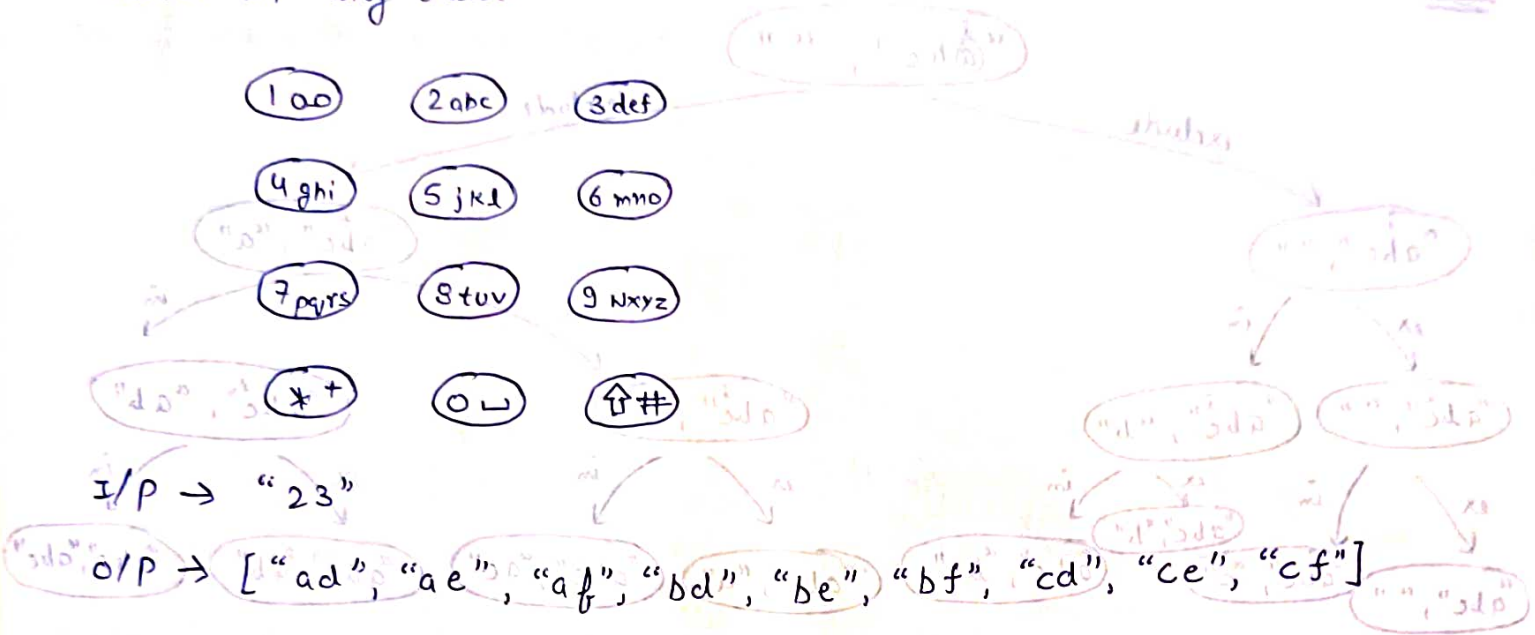
```
solve (str, output, index+1, ans);
```

H.O. Do previous 2 questions by Bit manipulation

Lecture 38

Lecture-17

Ques- Given a string containing digits from 2-9 inclusive, return all possible letter combinations that the number could represent. Return the answer in any order.

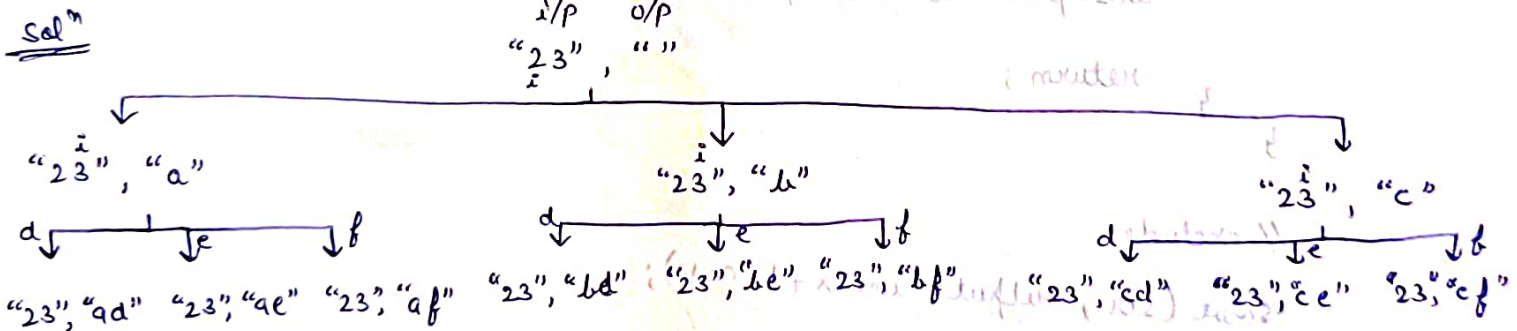


I/P → ""

O/P → []

I/P → "2"

O/P → ["a", "b", "c"]



```
vector<string> letterCombinations(string digits) {
    vector<string> ans;
    string output;
    int index = 0;
    string mapping[10] = {"", "", "abc", "def", "ghi", "jkl", "mno", "pqrs", "tuv", "wxyz"};
}
```

solve (digits, output, index, ans, mapping);

return ans;

}

Code for solve:-

void solve (string digit, string output, int index, vector<string> &ans, string mapping[]) {

// base case

if (index >= digit.length()) {

ans.push_back (output);

return;

}

int number = digit[index] - '0';

string value = mapping[number];

for (int i=0; i < value.length(); i++) {

output.push_back (value[i]);

solve (digit, output, index+1, ans, mapping);

output.pop_back(); Backtracking

}

}

Taaki dabara Ham apni
original state par pahuch

change is value ki string

store karne ke baad.

Lecture (39)

Same as lecture-46

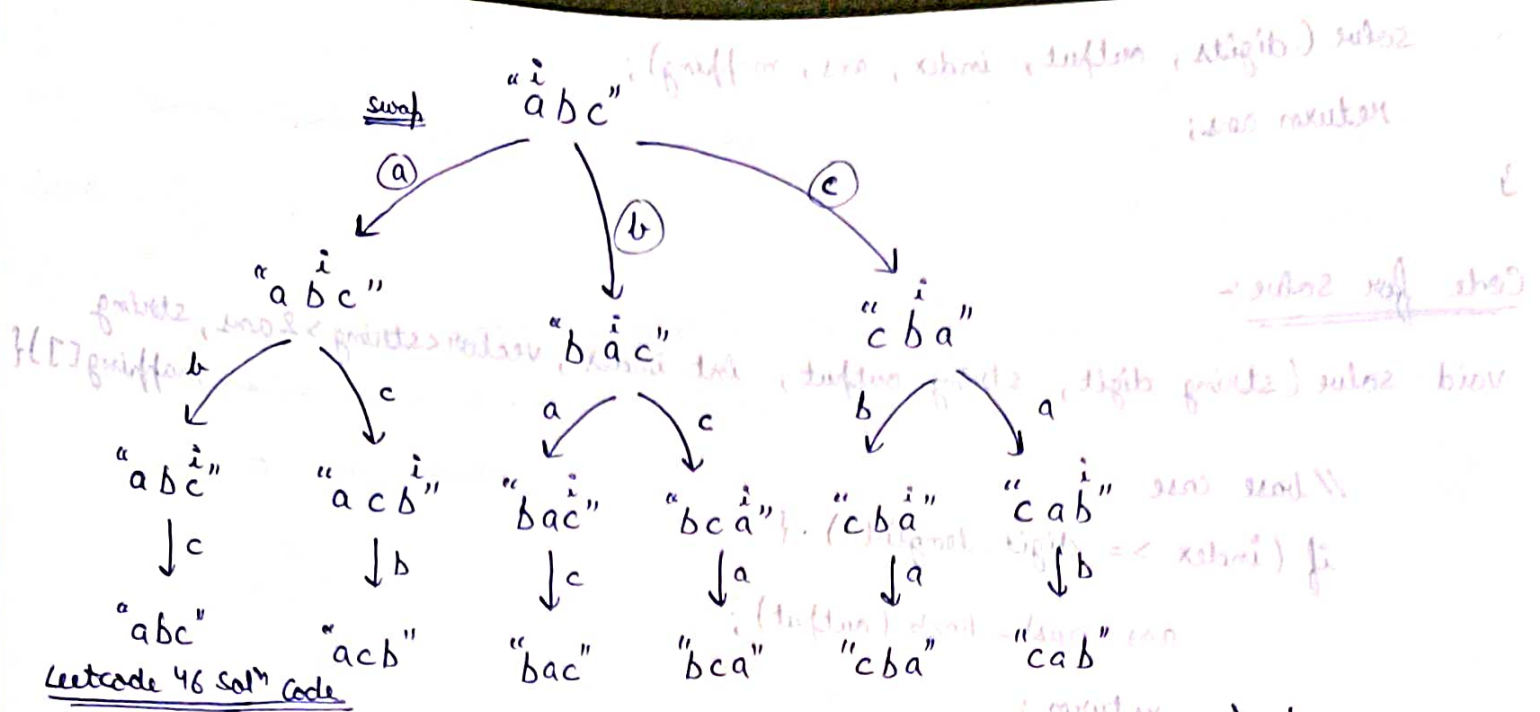
Q- You are given a string str consisting of lowercase English letters. Your

task is to return all permutations of the given string in lexicographically

increasing order.

i/p → abc

o/p → abc acb bac bca cab cba



Lecture 46 Soln Code

```

void solve (vector<int> nums, vector<vector<int>> &ans, int index) {
    if (index >= nums.size()) {
        ans.push_back(nums);
        return;
    }
    for (int j = index; j < nums.size(); j++) {
        swap (nums[index], nums[j]);
        solve (nums, ans, index+1);
        // Backtrack
        swap (nums[index], nums[j]);
    }
}
  
```

Lecture 40

Qu- Consider a rat placed at (0,0) in a square matrix of order $N \times N$. It has to reach the destination at $(N-1, N-1)$. Find all possible paths that the rat can take to reach from source to destination. The directions in which the rat can move are 'U' (up), 'D' (down), 'L' (left), 'R' (right). Value 0 at a cell in the matrix represents that

it is blocked and rat cannot move to it while value 1 at a cell in the matrix represents that rat can be travel through it. In a path, no cell can be visited more than one time. If the source cell is 0, the rat cannot move to any other cell.

i/p $\rightarrow N=4$

$m[i][j] = \{ \{1, 0, 0, 0\}, \{1, 1, 0, 1\}, \{1, 1, 0, 0\}, \{0, 1, 1, 1\} \}$

o/p $\rightarrow$ DDRDRR

i/p $\rightarrow N=2$

$m[i][j] = \{ \{1, 0\}, \{1, 0\} \}$

o/p $\rightarrow -1$

Approach

	0	1	2	3
0		↑		↓
1				
2	→			
3			←	

UP $\rightarrow (x-1, y)$

Down $\rightarrow (x+1, y)$

Right $\rightarrow (x, y+1)$

Left $\rightarrow (x, y-1)$

	0	1	2	3
0	1	0	0	0
1	1	1	0	1
2	1	1	0	0
3	0	1	1	1

Visited	0	1	2	3
0	✓	0	0	0
1	✓	0	0	0
2	✓	0	0	0
3	0	0	0	0

src $x = 0$

src $y = 0$

$(0,0) \rightarrow$ D/L/R/U

Down jaayega kya (3 cond check (i) 0 to nhi hai, (ii) Visited to nhi, (iii) Array ke Bachar to nhi)

Now, path = "D" & Visited $(1,0) = 1$

Ab firse poochenge down jaayega kya

Now, path = "DD" & Visited $(2,0) = 1$

Down jaayega kya $\rightarrow$ nhi (Because 0 hai)

Left jaayega kya $\rightarrow$ nhi (Array se Bachar Ho jaayega)

Right jaayega kya $\rightarrow$ Haa

Now, path = "DDR" & Visited (2,1) = 1

Down jayega Kya → Ha

path = "DDRD" & Visited (3,1) = 1

Down jayega Kya → Nahi (Array se Bahar Ho jaaunga)

Left Jayega Kya → Nahi (Because 0 hai)

Right Jayega Kya → Ha

Now, path = "DDRRD" & Visited (3,2) = 1

Down jayega Kya → Nahi (Array se Bahar Ho jaaunga)

Left Jayega Kya → Nahi (Because visited hai)

Right Jayega Kya → Ha

Now, final path = "DDRRRD"

When function call returns → Backtrack all visited calls

Code:-

```
void solve(vector<vector<int>> &m, int n, vector<string> &ans, int x, int y, vector<vector<int>> visited, string path) {
```

// Base case

if (x == n-1 & y == n-1) {

ans.push_back(path);

return;

visited[x][y] = 1;

// 4 choices -> D, L, R, U

// Down

int newX = x+1;

int newY = y;

Diagram showing a 4x4 grid with visited cells marked with 1. The path "DDRRRD" is highlighted with arrows: (0,0) → (1,0) → (2,0) → (2,1) → (2,2) → (3,2).

0	0	0	0
1	0	1	1
0	0	1	1
1	1	1	0

Diagram showing a 4x4 grid with visited cells marked with 1. The path "DDRRRD" is highlighted with arrows: (0,0) → (1,0) → (2,0) → (2,1) → (2,2) → (3,2).

0	0	0	0
1	0	1	1
0	0	1	1
1	1	1	0

Diagram showing a 4x4 grid with visited cells marked with 1. The path "DDRRRD" is highlighted with arrows: (0,0) → (1,0) → (2,0) → (2,1) → (2,2) → (3,2).

0	0	0	0
1	0	1	1
0	0	1	1
1	1	1	0

```

if (isSafe(newx, newy, n, visited, m)) {
    path.push-back('D');
    solve(m, n, ans, newx, newy, visited, path);
    path.pop-back();
}

```

// left

newx = x;

newy = y - 1;

```

if (isSafe(newx, newy, n, visited, m)) {

```

path.push-back('L');

solve(m, n, ans, newx, newy, visited, path);

path.pop-back();

}

// Right

newx = x;

newy = y + 1;

```

if (isSafe(newx, newy, n, visited, m)) {

```

path.push-back('R');

solve(m, n, ans, newx, newy, visited, path);

path.pop-back();

}

// Up

newx = x - 1;

newy = y;

```

if (isSafe(newx, newy, n, visited, m)) {

```

path.push-back('U');

solve(m, n, ans, newx, newy, visited, path);

path.pop-back();

}

```

    visited[x][y] = 0;
}

bool isSafe(int x, int y, int n, vector<vector<int>> visited, vector<vector<int>> m) {
    if ((x >= 0 && x < n) && (y >= 0 && y < n) && visited[x][y] == 0 && m[x][y] == 1) {
        return true;
    }
    else {
        return false;
    }
}

```

```

vector<string> findPath(vector<vector<int>> m, int n) {
    vector<string> ans;
    int srcx = 0;
    int srcy = 0;

    vector<vector<int>> visited = m;

    // initialize with 0
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            visited[i][j] = 0;
        }
    }

    string path = "";
    solve(m, n, ans, srcx, srcy, visited, path);
    sort(ans.begin(), ans.end());
    return ans;
}

```

Lecture 41

Time and Space Complexity of Recursive Algorithms

① Recursive code of Factorial

```
int fact(int n) {
    if (n == 0) {
        return 1;
    }
}
```

return $n * \text{fact}(n-1);$

Recurrence relⁿ $\Rightarrow F(n) = n * F(n-1)$

$\Rightarrow T(n) = K_1 + K_2 + T(n-1)$

Let $K_1 + K_2 = K$ (const.)

$T(n) = K + T(n-1)$

$T(n-1) = K + T(n-2)$

$T(n-2) = K + T(n-3)$

$T(1) = K + T(0)$

$T(0) = K_1$

$T(n) = n * K + K_1$

$T(n) = Kn$

$\Rightarrow T.C = O(n)$

(neglecting constant)

② Binary Search

$F(n) = \underbrace{xyz}_{\text{processing time}} + \underbrace{F(\frac{n}{2})}_{\text{array half ho raha hai}}$

Taking constant K_1 as base case complexity & K_2 as processing

$T(n) = K_1 + K_2 + T(n/2)$

$$\Rightarrow K_1 + K_2 = K \text{ (const.)}$$

$$T(n) = K + T(n/2)$$

$$T(n/2) = K + T(n/4)$$

$$T(n/4) = K + T(n/8)$$

$$\begin{array}{c} \vdots \\ T(2) = K + T(1) \\ T(1) = K \end{array}$$

$$T(n) = a * K$$

$$n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \rightarrow \dots \rightarrow 1 \quad \Rightarrow \quad \frac{n}{2^a} = 1$$

$$n = 2^a$$

$$a = \log n$$

$$\text{so, } T(n) = K \log n$$

Neglecting constant

$$\text{so, } T.C = O(\log n)$$

③ Merge sort

Steps

- Break 2 array left & right
- Recursion sort (left/right)
- new array (merge)
- copy new array content → original array

$$T(n) = \underbrace{K_1}_{\text{Basecase}} + \underbrace{K_2}_{\text{for mid}} + \underbrace{T(n/2)}_{\text{left part}} + \underbrace{T(n/2)}_{\text{right part}} + \underbrace{K_3 n}_{\text{merge}} + \underbrace{K_4 n}_{\text{copy}}$$

$$T(n) = K + 2T(n/2) + n(K_3 + K_4)$$

$$T(n) = 2T(n/2) + nK_5$$

$$T(n) = 2T(n/2) + nK$$

$$T(n) = 2T(n/2) + nK$$

$$2T(n/2) = 4T(n/4) + nK$$

$$4T(n/4) = 8T(n/8) + nK$$



$$T(1) = K$$

$$T(n) = a * nK$$

$$a = \log n$$

$$T(n) = Kn \log n$$

Neglecting constant

$$\Rightarrow T.C. = O(n \log n)$$

④ Fibonacci Series

$$T(n) = \underbrace{K_1}_{\text{Base case}} + \underbrace{K_2}_{\text{for add}} + T(n-1) + T(n-2)$$

$$T(n) = K + T(n-1) + T(n-2)$$

$$T(n-1) = K + T(n-2) + T(n-3)$$

$$T(n-2) = K + T(n-3) + T(n-4)$$

$$T(1) = K_1$$

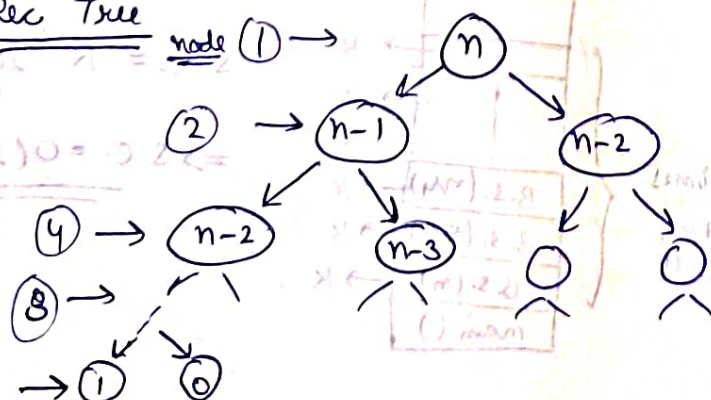
$$T(0) = K_1$$

But there are remaining
so this is h.w. that
if we can try finding
by this way, possible
or not.

Another way

Rec Tree

node ① →



each node = K time (let)

Total time = K * total node

$$\text{Total node} = 1 + 2^1 + 2^2 + \dots + 2^{n-1} + (2^n)T = (n)T$$

$$= 2^{n+1} - 1$$

$$T.C. = K * (2^{n+1} - 1)$$

$$= K * 2^{n+1} - K$$

$$T.C. = K * 2 * 2^n$$

$$\Rightarrow \underline{T.C. = O(2^n)} \quad (\text{exponential})$$

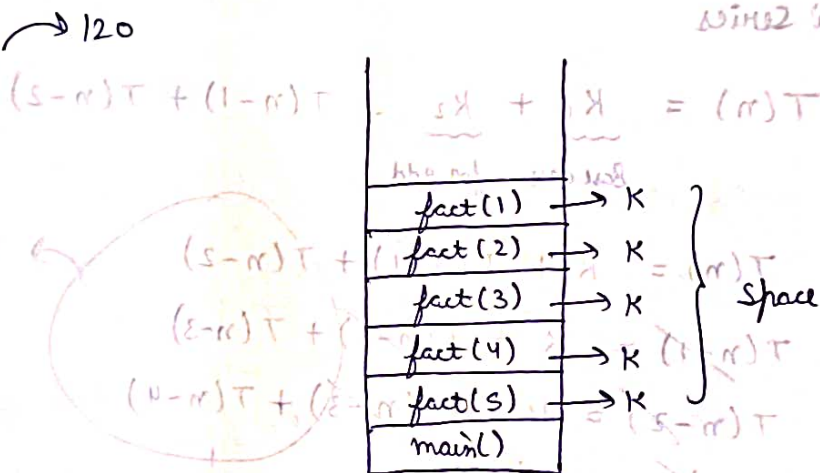
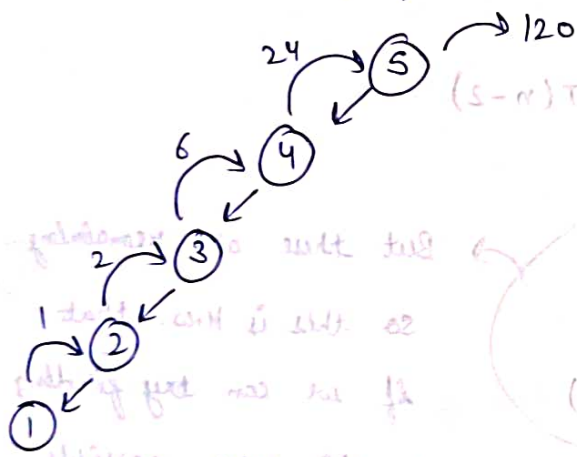
H.W.

Find T.C. of remaining questions of Recursion

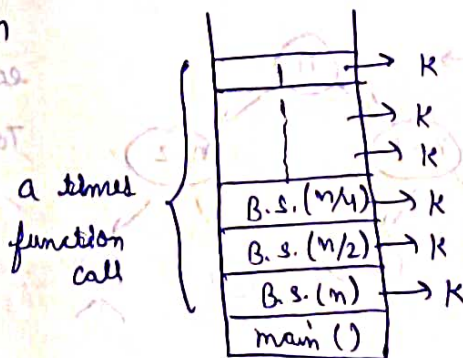
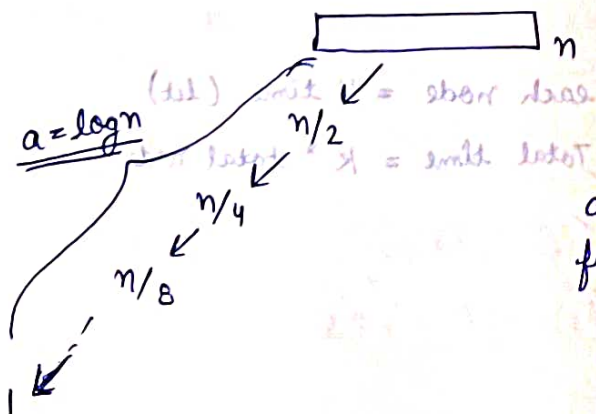
Space Complexity

① Factorial

let say for 5



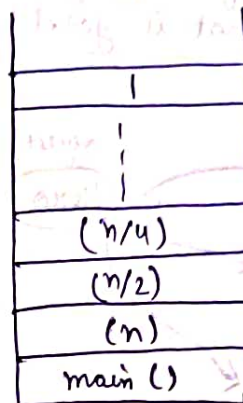
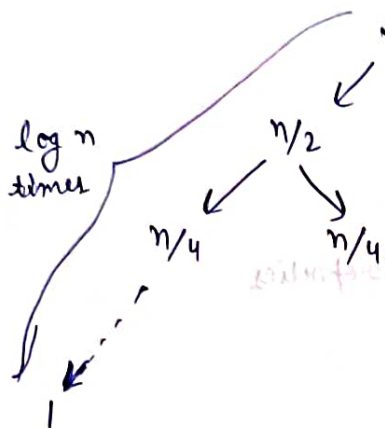
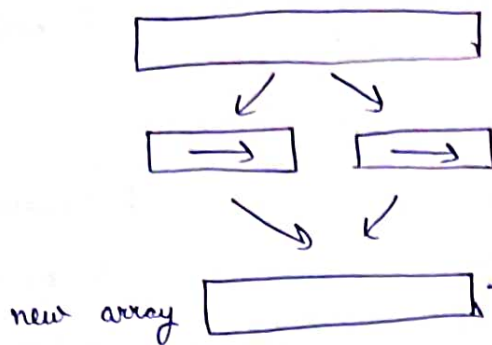
② Binary search



$$s.c. = K * \log n$$

$$\Rightarrow \underline{s.c. = O(\log n)}$$

③ Merge Sort

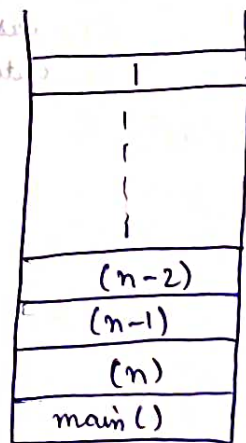
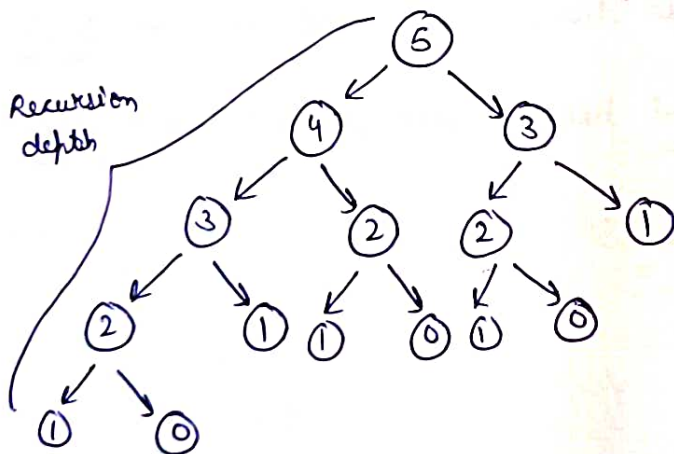


$$s.c. = K \log n + \frac{n}{\text{for new array}}$$

$$\& n \gg \log n$$

$$sa, s.c. = \underline{\underline{O(\log n)}}$$

④ Fibonacci Series



$$sa, s.c. = \underline{\underline{O(n)}}$$

H.W.

Find the s.c. of remaining questions of recursion & read 'code studio' complexity analysis